



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Utilizzo di algoritmi quantistici per ottimizzazione dell'esecuzione di scenario di simulazione per ADAS

RELATORE

Prof. Dario Di Nucci

Dott. Antonio Trovato

Università degli Studi di Salerno

CANDIDATO

Maksym Savitskyi

Matricola: 0512118179

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Abstract

Gli **Automated Driving Systems(ADS)** e gli **Advanced Driver Assistance Systems (ADAS)** sono fondamentali per la sicurezza e l'automazione dei veicoli moderni. Questi sistemi sono progettati per supportare o sostituire il conducente in diverse attività di guida, ridurre gli errori umani e aumentare il livello complessivo di sicurezza dei veicoli. Data la natura critica di tali applicazioni è fondamentale garantire un elevato livello di affidabilità del software che governa i comportamenti di questi sistemi. Tuttavia testare questi sistemi direttamente su strada risulta estremamente complesso, costoso e in molti casi potenzialmente pericoloso. Per questo motivo, nello sviluppo di sistemi **ADS** e **ADAS** viene spesso adottato il **Simulation-Based Testing**, che consente di valutare il comportamento del sistema all'interno di ambienti virtuali controllati in grado di riprodurre scenari realistici di guida. Attraverso la simulazione è possibile generare e ripetere numerosi scenari variando configurazione della strada e le condizioni ambientali. Nel caso di sistemi particolarmente complessi, l'utilizzo di un singolo simulatore può risultare insufficiente per rappresentare in modo accurato tutte le componenti del sistema e dell'ambiente, per questo motivo viene spesso adottato un approccio di **co-simulazione**, in cui più simulatori distinti vengono integrati tra loro per modellare diversi aspetti del sistema e dell'ambiente di guida. In questi contesti ambienti di simulazione come **CARLA** permettono di combinare diversi modelli, tra cui simulatori del traffico, delle condizioni meteorologiche e della dinamica dei veicoli, al fine di ottenere una rappresentazione più realistica degli scenari di guida. L'obiettivo di questa tesi è analizzare e confrontare tecniche di selezione di scenari all'interno di ambienti di simulazione e co-simulazione, al fine di migliorare l'efficacia del processo di testing dei sistemi **ADS** e **ADAS**. I risultati di questa tesi mostrano che l'algoritmo basato su quantum computing **IGDec** risulta migliore degli altri algoritmi considerati, in ogni ambiente simulato, suggerendo che sia molto promettente per la selezione di casi di test in sistemi **ADS** e **ADAS**.

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	iv
1 Introduzione	1
1.1 Contesto Applicativo	1
1.2 Motivazioni ed Obiettivi	1
1.3 Risultati	2
1.4 Struttura della Tesi	2
2 Background	4
2.1 Sistemi ADS e ADAS	4
2.1.1 Importanza del Testing nei Sistemi ADS e ADAS	5
2.1.2 Simulation-Based Testing	5
2.2 Selezione di Casi di Test con Algoritmi Quantistici	6
2.2.1 Quantum Computing	7
2.2.2 Algoritmi Quantistici per la Selezione di Casi di Test	8
2.3 Algoritmi Utilizzati	9
2.3.1 QAOA-TCS	9
2.3.2 IGDdec-QAOA	10
2.3.3 DIV-GA	12
2.3.4 Additional Greedy	13

3	Metodologia	15
3.1	Domanda di Ricerca	15
3.2	Scenari Utilizzati	15
3.2.1	Metriche Utilizzate	18
3.3	Configurazione Sperimentale	20
3.3.1	Esecuzione degli Algoritmi	20
3.3.2	Fronte di Pareto	20
3.3.3	Analisi Statistica	21
3.3.4	Processo di Confronto dei Fronti di Pareto	22
4	Analisi dei Risultati	24
4.1	Limiti del Lavoro	26
5	Conclusioni e Lavori Futuri	28
5.1	Conclusioni	28
5.2	Lavori Futuri	29
	Bibliografia	30

Elenco delle figure

3.1	Fronte di Pareto di IGDec nel simulatore Fake Brisbane vs Fronte di Pareto di Riferimento	22
-----	---	----

Elenco delle tabelle

4.1	Simulatore Ideale	24
4.2	Simulatore Sampling Noise	24
4.3	Simulatore Fake Brisbane	25
4.4	Simulatore Depolarizing con rumore di 1%	25
4.5	Simulatore Depolarizing con rumore di 2%	25
4.6	Simulatore Depolarizing con rumore di 5%	25

Introduzione

1.1 Contesto Applicativo

Negli ultimi anni il settore automobilistico ha avuto una crescente diffusione di sistemi **ADS** e **ADAS**. Queste tecnologie hanno l'obiettivo di migliorare la sicurezza sulla strada, ridurre il numero di incidenti e supportare il conducente nelle diverse attività di guida. Questi sistemi si basano su comportamenti software complessi che integrano informazioni provenienti da molteplici sensori come telecamere, radar e lidar, al fine di percepire l'ambiente circostante e prendere decisioni in tempo reale. A causa della natura critica di tali applicazioni, eventuali errori nel software possono avere conseguenze significative sulla sicurezza degli utenti della strada, per questo motivo, il processo di verifica e validazione assume ruolo fondamentale nello sviluppo di questi sistemi.

1.2 Motivazioni ed Obiettivi

La validazione dei sistemi di assistenza alla guida e dei sistemi di guida automatizzata rappresenta una delle principali sfide tecnologiche e scientifiche nel settore automobilistico. I veicoli devono essere in grado di operare in contesti complessi, dinamici, spesso poco prevedibili ed caratterizzati da un'elevata variabilità delle condizioni di traffico, dell'ambiente circostante e del comportamento degli altri partecipanti della strada. Talvolta, una delle principali difficoltà consiste proprio nel fatto

che il numero di possibili situazioni di guida è estremamente elevato. Garantire che un sistema di assistenza alla guida o sistema di guida automatizzata sia robusto e sicuro richiede quindi la valutazione su strada del suo comportamento in un numero molto ampio di situazioni possibili, tuttavia, testare tutte le possibili configurazioni dell'ambiente di guida è praticamente impossibile nella pratica, sia per motivi di tempo, sia per i costi associati ai processi di testing.

Questa complessità rende necessario sviluppare metodologie e strumenti che consentano di migliorare l'efficacia delle attività di verifica e validazione, permettendo di identificare situazioni rilevanti per il testing e di valutare in modo sistematico il comportamento del sistema in condizioni critiche.

1.3 Risultati

Da confronti dei fronti di Pareto tra gli algoritmi di Test Case Selection tradizionali e quelli basati sull'approccio quantistico, è emerso che l'algoritmo **IGDec**, risulta il migliore sia nelle simulazioni con ideali (senza rumore) sia negli ambienti con vari gradi di rumore. Questo risultato viene in seguito confermato dall'applicazione di vari test statistici. Nel complesso i risultati di questa tesi suggeriscono che gli approcci basati su tecniche di ottimizzazione quantistica, in particolare IGDec, rappresentano una direzione promettente per affrontare il problema della selezione degli scenari nel testing di sistemi ADS e ADAS.

1.4 Struttura della Tesi

La tesi è organizzata in cinque capitoli principali:

- **Capitolo 1 - Introduzione:** Definisce il contesto applicativo, le motivazioni e gli obiettivi e i risultati finali del lavoro.
- **Capitolo 2 - Background:** Spiega gli sistemi ADS e ADAS e loro caratteristiche principali, spiegando in seguito l'importanza del testing di questi sistemi, introducendo il concetto di Simulation-Based Testing. In seguito viene descritta la

metodologia di valutazione seguita dalla descrizione delle differenze di Test Case Selection Tradizionali dal Approccio Quantistico.

- **Capitolo 3 - Metodologia:** Descrive in maniera approfondita come sono composti gli scenari e le metriche utilizzate, illustrando anche la configurazione sperimentale e il ruolo dei fronti di Pareto.
- **Capitolo 4 - Analisi dei Risultati:** Dimostra gli risultati ottenuti insieme a una analisi sintetica, spiegando successivamente limitazioni e assunzioni adottate.
- **Capitolo 5 - Conclusioni e Lavori Futuri:** Propone possibili modi di evoluzione di questo lavoro nel futuro.

Background

2.1 Sistemi ADS e ADAS

In questo contesto è possibile distinguere due principali categorie di sistemi: **Advanced Driver Assistance Systems (ADAS)** e **Automated Driving Systems (ADS)** [1]. Anche se progettati per lo stesso scopo, questi sistemi differiscono per il grado di controllo che il sistema esercita sul veicolo.

Advanced Driver Assistance Systems (ADAS) Sono sistemi progettati per assistere il conducente durante la guida. In questo caso il controllo del veicolo rimane principalmente al conducente umano, mentre il sistema fornisce supporto attraverso funzionalità di monitoraggio e avviso o in alcuni casi anche un intervento limitato. Le funzionalità più comuni di questi sistemi sono [1]:

- **Lane Keeping Assistance** che aiuta il conducente a mantenere il veicolo all'interno della corsia.
- **Blind Spot Detection** che segnala la presenza dei veicoli negli angoli ciechi.
- **Adaptive Cruise Control** che regola automaticamente la velocità del veicolo mantenendo una distanza di sicurezza dal veicolo che precede.
- **Automatic Emergency Braking** che attiva la frenata in caso di rischio di collisione.

In questi sistemi il conducente rimane comunque responsabile del controllo del veicolo e deve essere pronto a intervenire in qualsiasi momento.

Automated Driving Systems (ADS) Rappresentano un livello più avanzato di automazione, in cui il sistema è in grado di assumere il controllo completo del veicolo in specifiche condizioni operative. Il livello di automatizzazione è comunemente classificato secondo la scala definita dalla **Society of Automotive Engineers (SAE)**, che distingue sei livelli di automatizzazione, dal livello 0 (nessuna automatizzazione) al livello 5 (automatizzazione completa).

2.1.1 Importanza del Testing nei Sistemi ADS e ADAS

Poiché i sistemi **ADS** e **ADAS** operano in contesti reali e possono influenzare direttamente la sicurezza stradale, è fondamentale garantire che tali sistemi funzionino correttamente in una vasta gamma di condizioni operative. A differenza di molti sistemi software tradizionali, i sistemi di guida devono essere in grado di gestire situazioni estremamente diverse, tra cui condizioni di traffico variabili, differenti configurazioni stradali e comportamenti imprevedibili degli altri utenti della strada. Per questo motivo il processo di testing e validazione assume un ruolo cruciale nello sviluppo di questi sistemi. Il testing deve essere in grado di verificare il comportamento del sistema in numerosi scenari di guida, inclusi casi limite e situazioni potenzialmente critiche. Tuttavia il numero di scenari di guida è estremamente elevato e rende impraticabile testare tutte le possibili combinazioni possibili, di conseguenza, è necessario selezionare in modo efficace un insieme di scenari rappresentativi che permettono di valutare il comportamento del sistema in condizioni significative.

2.1.2 Simulation-Based Testing

Il Simulation-Based Testing rappresenta una metodologia ampiamente utilizzata per la verifica e la validazione di sistemi complessi, in particolare nel contesto dei sistemi automobilistici avanzati. Questa tecnica consiste nell'utilizzo di ambienti di simulazione per riprodurre scenari realistici di funzionamento del sistema e valutare il comportamento del sistema stesso in diverse condizioni operative. Nel

caso dei sistemi **ADS** e **ADAS**, che devono operare in ambienti dinamici e altamente variabili, caratterizzati da numerose configurazioni possibili della strada, del traffico e delle condizioni ambientali. Testare questi sistemi esclusivamente in condizioni reali sarebbe estremamente complesso, costoso e in molti casi pericoloso. Il Simulation-Based Testing consente di superare queste limitazioni offrendo la possibilità di eseguire test in ambienti virtuali controllati. In questi ambienti è possibile riprodurre scenari di guida realistici includendo variabili come la posizione dei veicoli, la configurazione della strada, le condizioni di traffico e altri elementi dell'ambiente. In questo modo diventa possibile valutare il comportamento del sistema in un ampio insieme di situazioni senza la necessità di eseguire test fisici sulla strada. Un altro vantaggio di questo approccio è la possibilità di eseguire un numero elevato di test in modo riproducibile. Gli stessi scenari possono essere ripetuti più volte per analizzare il comportamento del sistema in condizioni identiche, facilitando l'identificazione di eventuali anomalie o comportamenti indesiderati. Inoltre la simulazione permette di includere situazioni critiche o rare, che sarebbero difficili o rischiose da riprodurre nel mondo reale, per esempio verificare il comportamento del sistema quando incontra un animale sulla strada.

Nel contesto del Simulation-Based testing è importante distinguere tra i concetti di **simulazione** e **co-simulazione**. Con il termine simulazione si indica l'esecuzione di un modello del sistema all'interno di un singolo ambiente di simulazione, in cui tutte le componenti del sistema e dell'ambiente circostante vengono rappresentate e gestite dallo stesso simulatore. La **co-simulazione** invece consiste nell'integrazione di più simulatori distinti che collaborano tra loro durante l'esecuzione della simulazione. Ogni simulatore è responsabile della modellazione di una specifica parte del sistema o dell'ambiente, mentre un meccanismo di sincronizzazione coordina lo scambio di dati tra i diversi simulatori durante l'esecuzione. Questo approccio è solitamente utilizzato per testare sistemi **ADS** e **ADAS**.

2.2 Selezione di Casi di Test con Algoritmi Quantistici

Negli ultimi anni l'emergere delle tecnologie di **Quantum Computing** [2] ha aperto nuove possibilità per affrontare problemi di ottimizzazione complessi, in par-

icolare, l'utilizzo di algoritmi quantistici è stato proposto come possibile approccio per migliorare l'esplorazione dello spazio delle soluzioni in problemi combinatori caratterizzati da elevata complessità computazionale. Nel contesto di **Test Case Selection** [3] il problema può essere formulato come un problema di ottimizzazione in cui è necessario individuare un sottoinsieme di casi di test che soddisfi determinati criteri di qualità. Poiché questo tipo di problema può essere ricondotto a classi di problemi computazionalmente difficili, l'utilizzo delle tecniche basate su computazione quantistica rappresenta una possibile direzione per migliorare l'efficienza delle tecniche di ricerca delle soluzioni.

2.2.1 Quantum Computing

Il **Quantum Computing** [2] rappresenta un paradigma di calcolo alternativo rispetto ai sistemi di calcolo classici. Mentre nei sistemi tradizionali l'informazione è rappresentata tramite **bit**, che possono assumere valori tra 0 e 1, nei sistemi quantistici l'unità fondamentale di informazione è il **qubit**. Un qubit può essere rappresentato come una combinazione lineare dei due stati base secondo la seguente espressione:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

Dove α e β rappresentano coefficienti complessi associati alla probabilità di osservare rispettivamente lo stato $|0\rangle$ o $|1\rangle$. Quando entrambi i coefficienti sono diversi da zero, il qubit si trova in uno stato di **sovrapposizione** (*superposition*) in cui entrambi gli stati sono simultaneamente possibili fino al momento della misurazione. Quando il valore viene misurato, lo stato quantistico collassa in uno dei due valori classici. Grazie a queste proprietà i sistemi quantistici consentono di rappresentare e manipolare simultaneamente molte possibili configurazioni del problema, permettendo potenzialmente una migliore esplorazione dello spazio delle soluzioni rispetto agli approcci tradizionali.

Attualmente esistono due paradigmi principali di computazione quantistica utilizzati per affrontare problemi di ottimizzazione:

Il primo è **Adiabatic Quantum Computing (AQC)** [4], nel quale il sistema quantistico viene inizializzato nello stato fondamentale di una funzione Hamiltoniana semplice. Attraverso un processo di evoluzione adiabatica il sistema viene gradualmente

trasformato fino a raggiungere una nuova Hamiltoniana il cui stato fondamentale rappresenta la soluzione del problema. Questo paradigma è alla base dei sistemi di **Quantum Annealing**.

Il secondo paradigma è **Gate Model Quantum Computing (GMQC)** [5], nel quale il calcolo viene effettuato applicando sequenze di porte quantistiche a un insieme di qubit. Queste porte rappresentano l'equivalente quantistico delle porte logiche nei sistemi classici e permettono di costruire circuiti quantistici in grado di eseguire algoritmi complessi. Questo modello è utilizzato nei dispositivi quantistici per utilizzo generico, ma è anche il paradigma adottato da uno degli algoritmi più rilevanti per ottimizzazione combinatoria chiamato **Quantum Approximate Optimization Algorithm (QAOA)** [6], che utilizza circuiti quantistici parametrizzati per approssimare la soluzione dei problemi difficili.

Molti problemi di ottimizzazione affrontati tramite algoritmi quantistici vengono formalizzati attraverso il modello **Quadratic Unconstrained Binary Optimization (QUBO)** [7] che rappresenta il problema come una funzione quadratica di variabili binarie, in forma generale può essere espresso come:

$$\min / \max y = x^T Q x$$

Dove x è un vettore di variabili binarie di decisione e Q è una matrice quadrata di coefficienti costanti. Questa formulazione consente di rappresentare diversi problemi combinatori in una forma compatibile con algoritmi di ottimizzazione quantistica.

2.2.2 Algoritmi Quantistici per la Selezione di Casi di Test

Il problema del **Test Case Selection (TCS)** consiste nell'individuare un sottoinsieme di casi di test da eseguire con l'obiettivo di massimizzare l'efficacia del processo di testing riducendo al contempo i costi associati all'esecuzione dei test.

In questo contesto, diversi lavori hanno proposto l'utilizzo di modelli di ottimizzazione basati su **Quadratic Unconstrained Binary Optimization (QUBO)** [7] e algoritmi quantistici per la selezione dei casi di test. Tra questi approcci, **SelectQA** [8] che utilizza tecniche di quantum annealing per formulare il problema della selezione dei test come un problema di ottimizzazione combinatoria risolvibile tramite hardware

quantistico. Questo approccio ha mostrato risultati promettenti rispetto ai metodi tradizionali. Un ulteriore contributo è rappresentato da **IGDec-QAOA**[9] che utilizza il QAOA combinato con la strategia di decomposizione guidata dall'impatto per affrontare problemi di dimensioni maggiori, rispetto alle capacità dei dispositivi quantistici attuali.

Più recentemente, **Trovato et al.** hanno proposto **QAOA-TCS**[10], un approccio basato sempre su QAOA per la selezione dei casi di test che utilizza tecniche di clustering per suddividere il problema in sotto problemi più piccoli. Questo metodo consente di ridurre la complessità del problema e di rendere l'ottimizzazione compatibile con le limitazioni hardware dei sistemi quantistici.

2.3 Algoritmi Utilizzati

Negli confronti studiati in questa tesi sono stati utilizzati diversi algoritmi che utilizzano approcci diversi. Ci sono algoritmi classici come quelli genetici o quelli basati sulla strategia greedy e algoritmi quantistici come QAOA-TCS o IGDec-QAOA[9]. In questo capitolo vengono spiegati le idee base di ogni algoritmo trattato. Il problema della selezione dei casi di test è stato ampiamente studiato nella letteratura [3] nell'ambito del **Search-Based Software Engineering**. Gli approcci tradizionali affrontano questo problema utilizzando diverse tecniche di ottimizzazione, tra cui algoritmi greedy, metaeuristiche e metodi di ottimizzazione multi-obiettivo.

2.3.1 QAOA-TCS

L'algoritmo Quantum Approximate Optimization Algorithm - Test Case Selection (QAOA-TCS) [11] è un approccio basato su QAOA, progettato per affrontare il problema della selezione dei casi di test nei processi di regression testing. L'obiettivo principale consiste nell'individuare un sottoinsieme ottimale di casi di test che consenta di mantenere una buona copertura del sistema riducendo allo stesso tempo il costo di esecuzione della suite di test. Nel contesto dello sviluppo software moderno, l'esecuzione completa di una suite di regression test può risultare estremamente

costosa, soprattutto per i sistemi di grandi dimensioni. Per questo motivo tecniche di test case selection (TCS) vengono utilizzate per individuare un sottoinsieme rappresentativo dei test, in grado di preservare l'efficacia del processo di verifica limitando il costo computazionale.

Per applicare il QAOA al problema della selezione dei casi di test, il problema viene formulato come un modello QUBO (Quadratic Unconstrained Binary Optimization), in questa formulazione ogni variabile binaria rappresenta l'inclusione o l'esclusione di un determinato caso di test nella suite finale.

L'obiettivo è selezionare un sottoinsieme di casi di test che rappresenti un compromesso ottimale tra tre criteri stabiliti: Costo di esecuzione dei test, Copertura di fault individuati in passato e Copertura delle istruzioni del programma. Questi obiettivi vengono integrati in una funzione di costo rappresentata tramite un Hamiltoniano, che il QAOA cerca di minimizzare durante il processo di ottimizzazione. In particolare modello include:

- un termine lineare che rappresenta il costo di esecuzione dei test.
- un termine che premia i test che hanno individuato fault in passato.
- un termine quadratico che penalizza la selezione di test ridondanti, che coprono le stesse istruzioni del programma.

Uno dei principali limitazioni dei computer quantistici attuali è il numero di qubit disponibili. Per rendere il problema compatibile con tale limitazione, QAOA-TCS utilizza una strategia di decomposizione basata su clustering. In particolare, i casi di test vengono raggruppati utilizzando l'algoritmo K-Means, considerando i tre criteri definiti prima. La dimensione dei cluster viene limitata per garantire che il numero di variabili del problema rimanga compatibile con il numero di qubit disponibili sul dispositivo quantistico.

2.3.2 IGDec-QAOA

L'algoritmo Impact-Guided Decomposition - Quantum Approximate Optimization Algorithm (IGDec-QAOA) [9] è un approccio ibrido classico-quantistico progettato per risolvere problemi di ottimizzazione combinatoria di grandi dimensioni.

L'algoritmo combina il Quantum Approximate Optimization Algorithm (QAOA) con una strategia di decomposizione guidata dall'impatto (IGDec) con l'obiettivo di rendere possibile la risoluzione di problemi che eccedono la capacità dei computer quantistici attualmente disponibili.

Il problema di ottimizzazione viene inizialmente formulato utilizzando una rappresentazione Ising, che consente di mappare le variabili decisionali del problema sui qubit del circuito quantistico. Tuttavia i dispositivi quantistici attuali dispongono un numero molto limitato di qubit e non possono quindi affrontare direttamente problemi con un numero elevato di variabili, per questo motivo, IGDec-QAOA introduce una strategia di decomposizione che suddivide il problema originale in sottoproblemi più piccoli, risolvibili tramite QAOA.

Algoritmo è composto da 3 fasi principali: Inizializzazione, ordinamento per impatto e ottimizzazione.

Inizializzazione Nella prima fase, a ciascuna variabile decisionale viene assegnato un valore iniziale casuale, generalmente rappresentato come una sequenza binaria. Questa configurazione iniziale rappresenta una soluzione candidata del problema e consente di calcolare il valore iniziale della funzione obiettivo (fitness).

Ordinamento per Impatto La seconda fase introduce il concetto di *impact value*, che misura l'influenza di ciascuna variabile sul valore della funzione obiettivo. Per calcolare l'impatto di una variabile, l'algoritmo modifica temporaneamente il valore di quella variabile e osserva la variazione della funzione obiettivo risultante. La differenza tra il valore della funzione obiettivo prima e dopo la modifica rappresenta l'impatto della variabile.

Una volta calcolato l'impatto per tutte le variabili, queste vengono ordinate in base al loro valore di impatto. Nel caso di problemi di minimizzazione, le variabili con impatto più basso vengono posizionate all'inizio della lista, poiché la loro modifica contribuisce maggiormente alla riduzione del valore della funzione obiettivo.

Ottimizzazione Nella terza fase viene applicata la strategia di decomposizione guidata all'impatto. Poiché il numero di qubit disponibili è limitato, l'algoritmo

seleziona un sottoinsieme di variabili attive, chiamato *active set*, la cui dimensione è compatibile con la capacità del dispositivo quantistico. Il QAOA viene quindi applicato solo a questo sottoinsieme di variabili, mentre le altre rimangono temporaneamente fissate ai valori correnti. In questo modo il problema complessivo viene risolto iterativamente attraverso una sequenza di sotto problemi più piccoli. Dopo ogni esecuzione di QAOA, la soluzione corrente viene aggiornata con i nuovi valori ottimizzati delle variabili attive. Il processo continua selezionando progressivamente nuovi sottoinsiemi di variabili fino a quando non vengono soddisfatte determinate condizioni di arresto, come il raggiungimento di numero massimo di iterazioni o l'assenza di ulteriori miglioramenti nella funzione obiettivo.

2.3.3 DIV-GA

L'algoritmo Diversity Oriented - Genetic Algorithm (DIV-GA) [11] è un algoritmo evolutivo progettato per affrontare problemi di selezione in cui è necessario individuare un insieme di elementi che massimizzi la diversità complessiva. L'approccio si basa sui principi degli algoritmi genetici, che simulano i meccanismi dell'evoluzione naturale per esplorare lo spazio delle possibili soluzioni e cercare di individuare le configurazioni ottimali.

Anche funzionamento di DIV-GA è suddiviso in più fasi che sono: codifica delle soluzioni, inizializzazione della popolazione, funzione di fitness, applicazione dei operatori genetici e iterazione evolutiva.

Codifica delle Soluzioni Nel DIV-GA (come nei algoritmi genetici classici) ogni soluzione candidata è rappresentata come un cromosoma, tipicamente codificato come una stringa binaria. In questa rappresentazione ciascun gene del cromosoma indica se un determinato elemento del dataset è incluso nella soluzione. Un valore pari a 1 indica che l'elemento è selezionato, mentre un valore pari a 0 indica che l'elemento non fa parte della soluzione.

Inizializzazione della Popolazione L'algoritmo parte generando una popolazione iniziale di soluzioni candidate. Ogni individuo della popolazione rappresenta una

possibile selezione di elementi e viene creato in modo casuale, rispettando però eventuali vincoli del problema, come il numero massimo di elementi selezionabili.

Funzione di Fitness La funzione di fitness misura la diversità complessiva degli elementi selezionati nella soluzione. In generale, la diversità può essere calcolata considerando le distanze tra le caratteristiche degli elementi selezionati oppure valutando la copertura delle diverse configurazioni presenti nel dataset.

Applicazione dei Operatori Genetici Dopo la valutazione della popolazione, l'algoritmo applica una serie di operatori genetici per generare nuove soluzioni. Gli operatori principali sono:

- **Selezione:** vengono scelti gli individui con fitness più elevata per partecipare alla generazione successiva.
- **Crossover:** due individui selezionati vengono combinati per generare nuovi individui, scambiando parti dei loro cromosomi.
- **Mutazione:** alcuni geni dei cromosomi vengono modificati casualmente per introdurre variabilità nella popolazione ed evitare la convergenza prematura verso soluzioni non ottimali.

Attraverso applicazione ripetuta di questi operatori, la popolazione evolve progressivamente verso soluzioni con valore di fitness sempre migliori.

Iterazione Evolutiva Queste fasi vengono ripetute per diverse generazioni; Ad ogni generazione la popolazione viene aggiornata con nuovi individui generati dagli operatori genetici e valutati tramite la funzione di fitness. Nel corso delle iterazioni le soluzioni tendono a migliorare progressivamente, consentendo all'algoritmo di individuare configurazioni che massimizzano la diversità degli elementi selezionati.

2.3.4 Additional Greedy

L'algoritmo Additional Greedy [12] è un approccio euristico utilizzato per affrontare il problema della selezione di test case nel contesto del testing del software.

L'algoritmo si basa su una strategia greedy, ossia su una procedura iterativa, che ad ogni passo seleziona una soluzione localmente migliore, secondo un determinato criterio di ottimizzazione.

L'algoritmo Greedy classico costruisce una soluzione iterativamente, partendo da un insieme di test vuoto. Ad ogni iterazione viene selezionato il test case che offre il maggiore valore della funzione obiettivo tra quelli ancora disponibili. Nel caso della selezione dei test questo approccio può essere utilizzato, ad esempio, per selezionare i test case che offre la *div score* migliore.

L'algoritmo Additional Greedy rappresenta una variante migliorata di questo approccio, che consiste nel selezionare, ad ogni iterazione, il test case che fornisce il maggior miglioramento locale rispetto alla soluzione corrente.

Nel contesto di questo lavoro, l'algoritmo Additional Greedy viene utilizzato per selezionare insiemi di scenari all'interno del dataset visto nel Capitolo 3.2. L'obiettivo è costruire un insieme di scenari che massimizzi metriche di qualità quali **diversità degli scenari (Div Score)** e la presenza di eventi critici (**Collisions**). L'algoritmo costruisce la soluzione in maniera incrementale, partendo dalla soluzione rappresentata da un insieme di scenari vuoto e, ad ogni iterazione, seleziona lo scenario che fornisce il maggior contributo addizionale rispetto alle metriche considerate, relativamente agli scenari già selezionati al interno del singolo cluster.

Un aspetto importante dell'algoritmo Additional Greedy è che il valore delle metriche viene ricalcolato ad ogni iterazione, considerando solo il contributo addizionale dello scenario rispetto alla soluzione già costruita, questo consente di evitare la selezione di scenari ridondanti che non migliorerebbero la significamene le metriche considerate.

Nel contesto di questo lavoro non era stato necessario applicare Additional Greedy per selezione gli scenari, questo era già stato fatto ed è disponibile nel repository github.¹ Sulla base di questo risultato era stato costruito il fronte di Pareto per Additional Greedy seguendo la stessa procedura degli altri algoritmi.

¹https://github.com/mariocelzo/adas_testing/blob/main/analysis_results/risultato/analysis_report.json (visitato il 20/01/2026)

Metodologia

3.1 Domanda di Ricerca

L'obiettivo di questa tesi è comparare algoritmi tradizionali e quantistici per la selezione di casi di test per sistemi ADAS e ADS. Per cui mi pongo la seguente domanda di ricerca:

Q RQ₁. *Quale algoritmo tra quelli tradizionali e quantistici è il migliore per la selezione di casi di test per sistemi ADAS e ADS*

3.2 Scenari Utilizzati

Ogni algoritmo trattato in questo lavoro era stato eseguito sullo stesso dataset di scenari¹. Ogni scenario è composto di numerosi campi che descrivono caratteristiche dell'attore, il tipo dello scenario, punto di un eventuale impatto (nel caso degli scenari con collisione), condizioni meteorologiche e l'ambiente in cui si svolge lo scenario. In totale sono stati utilizzati **497 scenari** per le esecuzioni degli algoritmi. Di seguito la descrizione di ogni campo di un singolo test:

- **event_type:** Tipo di evento registrato nello scenario (per esempio collisione).
- **timestamp:** Istante temporale in cui l'evento si verifica all'interno dello scenario.
- **actor_id:** Identificato univoco dell'attore principale coinvolto nell'evento.

¹https://github.com/mariocelzo/adas_testing/tree/main/simulation_output

- **actor_type**: Tipologia dell'attore principale coinvolto nell'evento (per esempio veicolo o pedone).
- **other_actor_id**: Identificatore univoco dell'altro attore coinvolto nell'evento.
- **other_actor_type**: Tipologia dell'altro attore coinvolto nell'evento.
- **impact_location**: Posizione nello spazio tridimensionale in cui si verifica l'evento di impatto.
 - **x**: Coordinata sull'asse **X** della posizione di impatto nello scenario.
 - **y**: Coordinata sull'asse **Y** della posizione di impatto nello scenario.
 - **z**: Coordinata sull'asse **Z** della posizione di impatto nello scenario.
- **town**: Identificatore della mappa in cui si svolge lo scenario simulato.
- **town_characteristics**: Insieme di informazioni che descrivono le caratteristiche strutturali della mappa utilizzata nello scenario.
 - **map_name**: Percorso della mappa simulata.
 - **traffic_lights**: Numero di semafori presenti nella mappa.
 - **approx_curves**: Numero approssimativo di curve presenti nella rete stradale della mappa.
 - **approx_junctions**: Numero approssimativo di incroci o intersezioni presenti nella rete stradale della mappa.
 - **approx_roads**: Numero approssimativo di segmenti stradali presenti nella rete stradale della mappa.
- **road_type_at_collision**: Tipologia di strada nel punto in cui si verifica l'evento (ad esempio strada urbana, incrocio o curva)
- **weather**: Insieme di parametri che descrivono le condizioni meteorologiche presenti durante lo scenario.
 - **cloudiness**: Livello di copertura nuvolosa nell'ambiente simulato.

- **precipitation**: Intensità delle precipitazioni (ad esempio pioggia).
- **precipitation_deposits**: Quantità di accumulo di acqua o residui di precipitazione sulla superficie stradale.
- **wind_intensity**: Intensità del vento nello scenario scenario.
- **fog_density**: Livello di densità della nebbia presente nello scenario.
- **sun_altitude_angle**: Angolo di elevazione del sole rispetto all'orizzonte, che influenza le condizioni di illuminazione dello scenario.

```
1  [
2    {
3      "event_type": "collision",
4      "timestamp": "1753966976.36",
5      "actor_id": 15709,
6      "actor_type": "vehicle.ford.mustang",
7      "other_actor_id": 15876,
8      "other_actor_type": "vehicle.carlamotors.firetruck",
9      "impact_location": {
10         "x": 334.7141418457031,
11         "y": 309.6476745605469,
12         "z": -0.00544752087444067
13     },
14     "town": "Town01",
15     "town_characteristics": {
16         "map_name": "Carla/Maps/Town01",
17         "traffic_lights": 36,
18         "approx_curves": 28,
19         "approx_junctions": 12,
20         "approx_roads": 98
21     },
22     "road_type_at_collision": "straight",
23     "weather": {
24         "cloudiness": 70.0,
25         "precipitation": 20.0,
26         "precipitation_deposits": 30.0,
27         "wind_intensity": 0.0,
28         "fog_density": 0.0,
29         "sun_altitude_angle": 0.0
30     }
31 }
32 ]
```

3.2.1 Metriche Utilizzate

Div Score [11] Misura di diversità di tutti gli scenari selezionati. Ogni scenario ha un valore di diversità assegnato tramite la procedura *compute_div_scores* (disponibile nello script *qrt_utility.py*). Per effettuare correttamente il calcolo, tutti gli attributi di un scenario vengono trasformati in rappresentazione numerica tramite il processo di *encoding*. Vengono applicati due tipi di encoding in base al tipo dell'attributo.

- Per gli attributi **categorici** viene applicato il *One-Hot Encoding*, rappresentando così ogni categoria come un valore binario, che non deve essere ulteriormente normalizzato (dominio è già $[0,1]$).
- Per gli attributi **numerici** invece viene applicata la normalizzazione tramite *Min-Max scaling*, che converte questi attributi in valori nel intervallo tra 0 e 1. Questo assicura che attributi differenti contribuiscono al **div score** nella stessa maniera, evitando così che attributi con valori numerici molto grandi non dominano la metrica.

Dopo il preprocessing iniziale, ogni scenario viene rappresentato come un **feature vector** composto di valori normalizzati e valori categorici trasformati. Tutti **feature vector** sono concatenati in una matrice:

$$X \in R^{n \times d}$$

dove: **n** il numero degli scenari e **d** il numero delle feature

La diversità tra gli scenari viene calcolata come distanza tra le vettori tramite algoritmo di **pairwise Manhattan** [13], che viene definito come:

$$D(i, j) = \sum_{k=1}^d |x_{ik} - x_{jk}|$$

dove **d** il numero delle feature e x_{ik} = valore della feature k nello scenario i . Il risultato è la seguente matrice di distanze:

$$D \in R^{n \times d}$$

In cui ogni istanza $D(i, j)$ rappresenta la distanza tra gli scenari i e j . Infine il **Div Score** viene calcolato come distanza media di un singolo scenario i rispetto a tutti gli altri scenari nel cluster, nella seguente maniera:

$$DivScore(i) = \frac{1}{n-1} \sum_{j \neq i} D(i, j)$$

dove n li numero degli scenari e $D(i, j)$ è la Distanza tra gli scenari i e j tramite **pairwise Manhattan**. Il valore di diversità più grande indica che lo scenario è più diverso da gli altri e contribuisce di più al valore di **div score** totale del cluster.

Collision [14] è un valore discreto $\in [0, 1]$ che indica se in un scenario occorre una collisione. Definizione formale:

$$collision = \begin{cases} 0 & \text{no, nello scenario non succede una collisione} \\ 1 & \text{sì, nello scenario succede una collisione} \end{cases}$$

Questa informazione non deve essere calcolata perché è già contenuta in ogni scenario (vedi capitolo 3.2). Il valore **collision** totale per ogni cluster viene calcolato in questo modo:

$$Collision = \sum_{i=1}^n collision_i$$

dove $collision_i \in [0, 1]$. Questa metrica misura la criticità di un scenario. Il valore di **collision** più alto indica che il cluster contiene più scenari critici.

Cost [3] rappresenta quantità di risorse necessarie per eseguire un scenario. Un assunzione che viene fatta in questo lavoro è che ogni singolo scenario ha un valore costante pari a **60**. Il costo totale di un cluster quindi è pari al numero di scenari selezionati moltiplicato per la costante. Formalmente:

$$Cost = 60 \times |S|$$

dove S = set degli scenari selezionati Questa metrica rappresenta l'efficienza. Valore più basso implica meno tempo di simulazione ed minori risorse computazionali

richieste. Ogni algoritmo ha come obbiettivo massimizzare **div score** e **collision** minimizzando nello stesso tempo il **costo**.

3.3 Configurazione Sperimentale

Questo capitolo descrive la configurazione sperimentale adottata per il confronto degli algoritmi analizzati in questa tesi. L'obbiettivo degli esperimenti è di valutare la qualità delle soluzioni dai diversi algoritmi di selezione attraverso un confronto basato su ottimizzazione multi-obiettivo.

3.3.1 Esecuzione degli Algoritmi

Considerando che alcuni algoritmi analizzati presentano un comportamento non deterministico, le loro esecuzioni possono produrre risultati differenti tra diverse iterazioni. Per tenere conto di questo aspetto, tutti gli algoritmi (ad eccezione Additional Greedy, che non possiede questa caratteristica) sono state fatte **10 esecuzioni** indipendenti e i risultati ottenuti sono stati analizzati considerando le distribuzioni dei valori ottenuti. Ogni fronte contiene un insieme di soluzioni candidate, ciascuna rappresentata tramite il vettore delle tre metriche descritte in precedenza.

3.3.2 Fronte di Pareto

Nel contesto dell'ottimizzazione multi-obiettivo, il confronto tra soluzioni viene effettuato utilizzando il concetto di **Fronte di Pareto**. Una soluzione è considerata *dominante* rispetto ad un'altra se è almeno altrettanto buona in tutte le metriche considerate ed è strettamente migliore in almeno una di esse. Una soluzione invece si dice *non dominata* quando non esiste un'altra soluzione che la domini rispetto a tutte le metriche considerate. L'insieme delle soluzioni non dominate costituisce la Fronte di Pareto che rappresenta l'insieme delle migliori soluzioni tridimensionalmente definito nelle metriche di costo, diversità e collisioni.

Per poter confrontare in modo uniforme i risultati prodotti dai diversi algoritmi è stata costruita una **Fronte di Pareto di Riferimento**. Questa fronte viene ottenuta considerando l'unione di tutte le soluzioni prodotte dagli algoritmi analizzati. Per

ogni soluzione generata, viene verificato se essa sia dominata da un'altra soluzione appartenente ad uno degli altri algoritmi. Le soluzioni che non risultano dominate da nessun'altra vengono incluse nella fronte di Pareto di riferimento. Questo processo consente di individuare l'insieme delle migliori soluzioni globali ottenute durante l'intero processo sperimentale.

Una volta costruito il fronte di Pareto di riferimento viene calcolato, per ciascun algoritmo il numero di soluzioni appartenenti alle proprie frontiere che risultano presenti nella fronte di riferimento. Questo valore rappresenta il numero di soluzioni non dominate prodotte da ciascun algoritmo.

Il confronto tra gli algoritmi viene quindi effettuato analizzando la distribuzione del numero di soluzioni non dominate prodotte nelle diverse esecuzioni

3.3.3 Analisi Statistica

Per verificare la significatività statistica delle differenze osservate tra gli algoritmi, i risultati ottenuti sono stati analizzati utilizzando test statistici non parametrici, in particolare, sono stati applicati il test di Kruskal-Wallis [15] per verificare la presenza di differenze significative tra i gruppi e il test di Dunn [16] con la correzione di Bonferroni per effettuare confronti pairwise tra gli algoritmi. Inoltre è stata utilizzata la misura di effect size di Vargha-Delaney (A_{12}) [17] per valutare la dimensione dell'effetto delle differenze osservate tra le distribuzioni dei risultati.

Questa analisi statistica consente di valutare in modo rigoroso le prestazioni relative degli algoritmi considerati nel contesto del problema di selezione degli scenari.

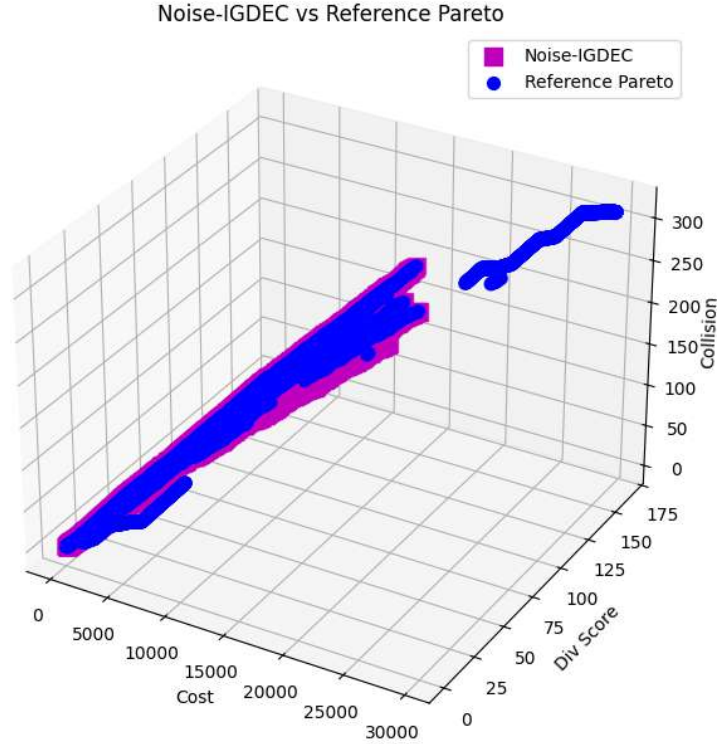


Figura 3.1: Fronte di Pareto di IGDec nel simulatore Fake Brisbane vs Fronte di Pareto di Riferimento

3.3.4 Processo di Confronto dei Fronti di Pareto

Il processo sperimentale è organizzato in **6 iterazioni**, ciascuna corrispondente ad una diversa configurazione di simulazione utilizzata per l'esecuzione degli algoritmi quantistici. Le configurazioni considerate includono diversi simulatori e modelli di rumore, quali sono: Simulatore Ideale [18], Sampling Noise [19], Fake Brisbane [19] e Depolarizing [19] con la percentuale di rumore pari a 1%, 2%, 5%.

Per ciascuna iterazione vengono raccolti i fronti di Pareto prodotte dai diversi algoritmi analizzati. Come detto in precedenza, gli algoritmi stocastici vengono eseguiti per **10 volte**, generando un insieme di fronti di Pareto indipendenti. Complessivamente il

processo sperimentale produce **31 fronti di Pareto** per ogni iterazione, corrispondenti alle diverse esecuzioni degli algoritmi considerati.

Una volta raccolte tutte le soluzioni generate, viene costruita una fronte di pareto di riferimento, che viene ottenuta combinando tutte le soluzioni prodotte dai diversi algoritmi e identificando quelle che risultano non dominate rispetto alle altre. Successivamente per ciascun algoritmo viene calcolato il numero di soluzioni appartenenti alle proprie fronti che risultano presenti nella fronte di Pareto di riferimento

Capitolo

4

Analisi dei Risultati

In questo capitolo vengono analizzati i risultati degli confronti, approfondendo gli risultati di applicazione degli test e metriche descritte nel precedente. Per ogni configurazione era stato eseguito il Test di Kruskal-Wallis H, ma per ogni uno il $p - \text{valore}$ era sempre < 0.5 . Analogamente il Test di Shapiro-Wilk [20] per ogni configurazione produce il valore "non normalizzato".

Tabella 4.1: Simulatore Ideale

Ipotesi	$p - \text{value}$	$p - \text{value}$ aggiustato	$\hat{A}_{12}$ Effect Size
IGDEC > QAOA-TCS	< 0.01	< 0.01	1.0
IGDEC > Additional-Greedy	< 0.01	< 0.01	1.0
IGDEC > DIV-GA	< 0.01	< 0.01	1.0

Tabella 4.2: Simulatore Sampling Noise

Ipotesi	$p - \text{value}$	$p - \text{value}$ aggiustato	$\hat{A}_{12}$ Effect Size
IGDEC > QAOA-TCS	< 0.01	< 0.01	0.9
IGDEC > Additional-Greedy	< 0.01	< 0.01	1.0
IGDEC > DIV-GA	< 0.01	< 0.01	1.0

Tabella 4.3: Simulatore Fake Brisbane

Ipotesi	$p - value$	$p - value$ aggiustato	$\hat{A}_{12}$ Effect Size
IGDEC > QAOA-TCS	< 0.01	< 0.01	0.9
IGDEC > Additional-Greedy	< 0.01	< 0.01	1.0
IGDEC > DIV-GA	< 0.01	< 0.01	1.0

Tabella 4.4: Simulatore Depolarizing con rumore di 1%

Ipotesi	$p - value$	$p - value$ aggiustato	$\hat{A}_{12}$ Effect Size
IGDEC > QAOA-TCS	< 0.01	< 0.01	0.9
IGDEC > Additional-Greedy	< 0.01	< 0.01	1.0
IGDEC > DIV-GA	< 0.01	< 0.01	1.0

Tabella 4.5: Simulatore Depolarizing con rumore di 2%

Ipotesi	$p - value$	$p - value$ aggiustato	$\hat{A}_{12}$ Effect Size
IGDEC > QAOA-TCS	< 0.01	< 0.01	0.9
IGDEC > Additional-Greedy	< 0.01	< 0.01	1.0
IGDEC > DIV-GA	< 0.01	< 0.01	1.0

Tabella 4.6: Simulatore Depolarizing con rumore di 5%

Ipotesi	$p - value$	$p - value$ aggiustato	$\hat{A}_{12}$ Effect Size
IGDEC > QAOA-TCS	< 0.01	< 0.01	0.9
IGDEC > Additional-Greedy	< 0.01	< 0.01	1.0
IGDEC > DIV-GA	< 0.01	< 0.01	1.0

Dai risultati degli test statistici, riportati nelle tabelle emerge che diverse coppie di algoritmi mostrano differenze statisticamente significative, evidenziate da $p - value$ corretti inferiori alla soglia di significatività stabilita come ≤ 0.05 . In particolare, i confronti che coinvolgono l'algoritmo IGDEC presentano valori di significatività molto elevati, con $p - value$ inferiori a 0.01 sia prima che dopo la correzione di Bonferroni, questo indica che le prestazioni di IGDEC risultano significativamente diverse rispetto a quelle degli altri algoritmi confrontati.

La misura di effect Vargha-Delaney A12 conferma tale risultato. I confronti tra IGDEC e gli altri algoritmi presentano valori di A12 pari a 1.0, indicando che in tutti i confronti effettuati, i valori prodotti da IGDEC risultano sistematicamente superiori a quelli degli altri algoritmi. Questo valore rappresenta la probabilità pari a 100% che il risultato prodotto da IGDEC sia maggiore di uno ottenuto da un altro algoritmo, suggerendo una differenza notevole nella distribuzione dei risultati.

Per quanto riguarda il confronto tra DIV-GA e gli altri algoritmi, si osserva un comportamento opposto. In particolare il valore di A12 pari a 0.0 nei confronti con QAOA-TCS e Additional-Greedy implica che i risultati ottenuti da DIV-GA risultano sistematicamente inferiori rispetto a quelli prodotti da gli altri due algoritmi. In questo caso il valore rappresenta la probabilità pari a 0%, che anche in questo caso è un valore estremo, suggerendo ancora una volta una differenza molto netta nella distribuzione degli risultati. Si nota anche che algoritmo Additional-Greedy non risulta superiore in nessuno degli confronti riportati.

🔗 **Answer to RQ₁.** Dagli confronti effettuati emerge che l'algoritmo quantistico **IGDec** è il algoritmo migliore per il problema di selezione di casi di test per sistemi ADAS e ADS, tra gli algoritmi quantistici e tradizionali confrontati

4.1 Limiti del Lavoro

Nel processo di valutazione e confronto degli algoritmi considerati in questa tesi sono state adottate alcune assunzioni e semplificazioni.

Numero degli scenari limitato: L'analisi sperimentale è stata condotta utilizzando un insieme di **497 scenari**. Questo numero consente di effettuare un confronto

preliminare tra gli algoritmi considerati, ma non rappresenta un insieme sufficientemente ampio per trarre conclusioni definitive sulle prestazioni degli algoritmi in contesti più complessi. In applicazioni reali il numero di scenari possibili può essere significativamente maggiore e includere una varietà più ampia di configurazioni dell'ambiente di guida.

Costo: Nel contesto degli esperimenti condotti, il costo associato all'esecuzione di ciascuno scenario è stato considerato costante e pari a **60**. Questa assunzione semplifica il problema di ottimizzazione, ma non riflette situazioni reali in cui diversi scenari possono richiedere tempi di esecuzione o risorse computazionali differenti.

Non Determinismo: Alcuni degli algoritmi confrontati presentano un comportamento non deterministico, poiché utilizzano meccanismi stocastici durante il processo di ottimizzazione, di conseguenza, i risultati prodotti da tali algoritmi possono variare tra diverse esecuzioni. Per mitigare questo effetto ogni algoritmo non deterministico è stato eseguito **10 volte**.

Conclusioni e Lavori Futuri

5.1 Conclusioni

In questa tesi è stato analizzato il problema della selezione degli scenari nel contesto di **Simulation-Based Testing** per sistemi **ADS** e **ADAS**, in particolare, sono stati confrontati diversi algoritmi di selezione degli scenari appartenenti sia alla categoria degli approcci tradizionali sia a quella degli approcci basati su tecniche di ottimizzazione quantistica.

Dalla applicazione degli test statistici sulle frontiere di Pareto generati in diversi ambienti simulati, con diversi gradi di rumore quantistico, emerge che l'algoritmo **IGDec** risulta il migliore in tutte le configurazioni sperimentali analizzate. Questo approccio ottiene consistentemente risultati strettamente migliori rispetto agli algoritmi Additional-Greedy e DIV-GA, essendo un po' meno performante di QAOA-TCS in ambienti con rumore.

Nel complesso i risultati di questa tesi suggeriscono che gli approcci basati su tecniche di ottimizzazione quantistica, in particolare IGDec, rappresentano una direzione promettente per affrontare il problema della selezione degli scenari nel testing di sistemi ADS e ADAS. Tuttavia ulteriori studi saranno necessari per valutare il comportamento di questi algoritmi su dataset di scenari più ampi e complessi.

5.2 Lavori Futuri

I risultati ottenuti in questa tesi aprono diverse possibili direzioni di ricerca che potrebbero essere esplorate in lavori futuri al fine di estendere e migliorare l'analisi condotta.

Una prima possibile estensione riguarda l'utilizzo di un numero significativamente maggiore di scenari di test. L'insieme di scenari utilizzati in questo lavoro, rappresenta solo una piccola porzione delle possibili configurazioni che possono emergere in ambienti di guida reali. L'integrazione di dataset più ampi e diversificati permetterebbe di valutare il comportamento degli algoritmi in condizioni più realistiche e di ottenere risultati statisticamente più robusti.

Ulteriori sviluppi potrebbero riguardare la definizione di modelli di costo più realistici per gli scenari di simulazione e miglioramento della definizione delle penalità, che potrebbe influenzare in modo molto significativo il comportamento degli algoritmi di ottimizzazione, e diverse configurazioni potrebbero portare a risultati differenti in termini di qualità delle soluzioni ottenute.

Bibliografia

- [1] O.-R. A. D. O. Committee, *Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*, Jun. 2018. [Online]. Available: https://doi.org/10.4271/J3016_201806 (Citato a pagina 4)
- [2] A. Steane, “Quantum computing,” *Reports on Progress in Physics*, vol. 61, no. 2, pp. 117–173, 1998. (Citato alle pagine 6 e 7)
- [3] S. Yoo and M. Harman, “Regression testing minimization, selection and prioritization: a survey,” *Software Testing, Verification and Reliability*, vol. 22, no. 2, pp. 67–120, 2012. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.430> (Citato alle pagine 7, 9 e 19)
- [4] T. Albash and D. A. Lidar, “Adiabatic quantum computation,” *Rev. Mod. Phys.*, vol. 90, p. 015002, Jan 2018. [Online]. Available: <https://link.aps.org/doi/10.1103/RevModPhys.90.015002> (Citato a pagina 7)
- [5] L. Gyongyosi and S. Imre, “Scalable distributed gate-model quantum computers,” *Scientific reports*, vol. 11, no. 1, p. 5172, 2021. (Citato a pagina 8)
- [6] J. Choi and J. Kim, “A tutorial on quantum approximate optimization algorithm (qaoa): Fundamentals and applications,” in *2019 international conference on information and communication technology convergence (ICTC)*. IEEE, 2019, pp. 138–142. (Citato a pagina 8)
- [7] F. Glover, G. Kochenberger, and Y. Du, “A tutorial on formulating and using qubo models,” *arXiv preprint arXiv:1811.11538*, 2018. (Citato a pagina 8)

-
- [8] A. Trovato, M. De Stefano, F. Pecorelli, D. Di Nucci, and A. De Lucia, "Reformulating regression test suite optimization using quantum annealing-an empirical study," *International Journal on Software Tools for Technology Transfer*, vol. 26, no. 6, pp. 767–780, 2024. (Citato a pagina 8)
- [9] X. Wang, S. Ali, T. Yue, and P. Arcaini, "Quantum approximate optimization algorithm for test case optimization," *IEEE Trans. Softw. Eng.*, vol. 50, no. 12, p. 3249–3264, Dec. 2024. [Online]. Available: <https://doi.org/10.1109/TSE.2024.3479421> (Citato alle pagine 9 e 10)
- [10] A. Trovato, M. Beseda, and D. Di Nucci, "A preliminary investigation on the usage of quantum approximate optimization algorithms for test case selection," in *Proceedings of the 2025 29th International Conference on Evaluation and Assessment in Software Engineering Companion*, ser. EASE Companion '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 56–60. [Online]. Available: <https://doi.org/10.1145/3727967.3756821> (Citato a pagina 9)
- [11] A. Panichella, R. Oliveto, M. D. Penta, and A. De Lucia, "Improving multi-objective test case selection by injecting diversity in genetic algorithms," *IEEE Transactions on Software Engineering*, vol. 41, no. 4, pp. 358–383, 2015. (Citato alle pagine 9, 12 e 18)
- [12] S. Yoo and M. Harman, "Pareto efficient multi-objective test case selection," in *Proceedings of the 2007 International Symposium on Software Testing and Analysis*, ser. ISSTA '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 140–150. [Online]. Available: <https://doi.org/10.1145/1273463.1273483> (Citato a pagina 13)
- [13] D.-J. Chang, A. H. Desoky, M. Ouyang, and E. C. Rouchka, "Compute pairwise manhattan distance and pearson correlation coefficient of data points with gpu," in *2009 10th ACIS International conference on software engineering, artificial intelligences, networking and parallel/distributed computing*. IEEE, 2009, pp. 501–506. (Citato a pagina 18)

- [14] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "Carla: An open urban driving simulator," in *Conference on robot learning*. PMLR, 2017, pp. 1–16. (Citato a pagina 19)
- [15] W. H. Kruskal and W. A. Wallis, "Use of ranks in one-criterion variance analysis," *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 583–621, 1952. [Online]. Available: <http://www.jstor.org/stable/2280779> (Citato a pagina 21)
- [16] O. J. Dunn, "Multiple comparisons using rank sums," *Technometrics*, vol. 6, no. 3, pp. 241–252, 1964. [Online]. Available: <https://www.tandfonline.com/doi/abs/10.1080/00401706.1964.10490181> (Citato a pagina 21)
- [17] A. Vargha and H. D. Delaney, "A critique and improvement of the "cl" common language effect size statistics of mcgraw and wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000. [Online]. Available: <http://www.jstor.org/stable/1165329> (Citato a pagina 21)
- [18] G. Aleksandrowicz, T. Alexander, P. Barkoutsos, L. Bello, Y. Ben-Haim, D. Bucher, F. J. Cabrera-Hernández, J. Carballo-Franquis, A. Chen, C.-F. Chen, J. M. Chow, A. D. Córcoles-Gonzales, A. J. Cross, A. Cross, J. Cruz-Benito, C. Culver, S. D. L. P. González, E. D. L. Torre, D. Ding, E. Dumitrescu, I. Duran, P. Eendebak, M. Everitt, I. F. Sertage, A. Frisch, A. Fuhrer, J. Gambetta, B. G. Gago, J. Gomez-Mosquera, D. Greenberg, I. Hamamura, V. Havlicek, J. Hellmers, Łukasz Herok, H. Horii, S. Hu, T. Imamichi, T. Itoko, A. Javadi-Abhari, N. Kanazawa, A. Karazeev, K. Krsulich, P. Liu, Y. Luh, Y. Maeng, M. Marques, F. J. Martín-Fernández, D. T. McClure, D. McKay, S. Meesala, A. Mezzacapo, N. Moll, D. M. Rodríguez, G. Nannicini, P. Nation, P. Ollitrault, L. J. O’Riordan, H. Paik, J. Pérez, A. Phan, M. Pistoia, V. Prutyanov, M. Reuter, J. Rice, A. R. Davila, R. H. P. Rudy, M. Ryu, N. Sathaye, C. Schnabel, E. Schoute, K. Setia, Y. Shi, A. Silva, Y. Siraichi, S. Sivarajah, J. A. Smolin, M. Soeken, H. Takahashi, I. Tavernelli, C. Taylor, P. Taylour, K. Trabing, M. Treinish, W. Turner, D. Vogt-Lee, C. Vuillot, J. A. Wildstrom, J. Wilson, E. Winston, C. Wood, S. Wood, S. Wörner, I. Y. Akhalwaya, and C. Zoufal, "Qiskit: An

- open-source framework for quantum computing,” Jan. 2019. [Online]. Available: <https://doi.org/10.5281/zenodo.2562111> (Citato a pagina 22)
- [19] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, “Quantum computing with Qiskit,” 2024. (Citato a pagina 22)
- [20] P. Royston, “Approximating the shapiro-wilk w-test for non-normality,” *Statistics and computing*, vol. 2, no. 3, pp. 117–119, 1992. (Citato a pagina 24)